Complete MLOps-DSPM Project Report

Development and Operationalization of Data Science Solutions

E-Commerce Shipping Prediction

Purpose	Explain what was implemented, where it is implemented, and why each activity is needed.
Process model	MLOps-based Data Science Process Model (MLOps-DSPM)
Final status	LIVE PROTOTYPE DEPLOYED - 2 OF 5 EVALUATION GATES MET
Live demo	https://huggingface.co/spaces/VivekSinghK14/shipping-delay-predictor

Reference requirements: DSPM_material_STP.pdf and STP_ecommerce.pdf

Executive summary

This project predicts, at the moment an order is placed, whether it will be **Delivered** on time or **Delayed**, and builds the complete MLOps lifecycle around that question: requirements, data pipeline, feature store, tracked experiments, model registry, evaluation, a deployed API and web app, monitoring, and retraining. Of the 50,000 orders in the source file, the 2,433 Returned orders belong to a different business process and were removed first; the remaining 47,567 orders are used throughout.

Result in one paragraph. The selected XGBoost model is live as a prototype (https://huggingface.co/spaces/VivekSinghK14/shipping-delay-predictor). On 9,514 test orders that were never used for training or tuning, it finds 47.4% of the delayed orders and reduces the estimated cost of delays by EUR 31,328 compared with doing nothing. It meets 2 of 5 agreed evaluation gates. Its ROC-AUC of 0.489 shows that the information available at order time (region, product category, shipping mode, cost, and date) does not yet separate delayed from delivered orders reliably, so the next iteration focuses on collecting operational data such as carrier, warehouse, route, and promised delivery date.

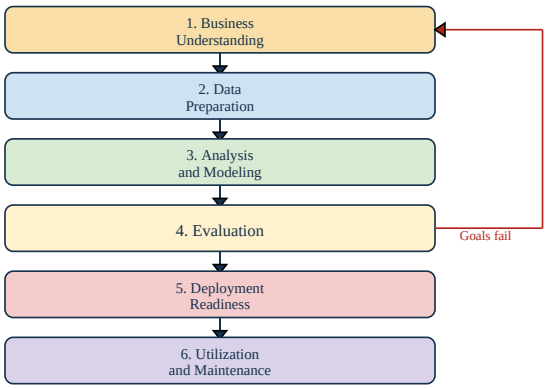
Key numbers at a glance

Item	Value	Section
Orders in source file / Returned removed / used	50,000 / 2,433 / 47,567	4
Delivered vs Delayed	40,062 (84.2%) vs 7,505 (15.8%)	4
Train / validation / test orders	28,539 / 9,514 / 9,514	4
Models compared	Naive baseline, Logistic Regression, Random Forest, XGBoost	5
Selected model and decision threshold	XGBoost, Delivered if P(Delivered) >= 0.85	5, 7
Test ROC-AUC / delayed recall / weighted F1	0.489 / 0.474 / 0.572	7
Estimated net benefit vs baseline	EUR -72,726 vs EUR -104,054	8
Evaluation gates met	2 of 5	7
Automated tests	24 passing	12

How to read this report

Each section covers one lifecycle stage, explains what we did and why, and ends with a **Where to find it** line naming the file behind every number.

Lifecycle at a glance



1. Requirements and process-model interpretation

The project proposal asks for a complete data science use case: business understanding, data preparation, model selection and evaluation, deployment, use, monitoring and maintenance, technical documentation, architecture diagrams, and a critical reflection. The MLOps-DSPM material describes who is responsible at each stage, which decisions must be made, which documents are produced, and which MLOps components (feature store, model registry, serving and monitoring) support the work. We used both documents as our checklist; section 13 maps every requirement to its evidence.

Scope decision

The question is binary: Delivered (1) or Delayed (0). Returned orders (2,433 rows, 4.9% of the file) are handled by a separate reverse-logistics process with different causes and different actions, so mixing them into the target would blur the question. They are removed in the first pipeline step and never reach features, models, reports, or the app.

A model only influences real business decisions after the evaluation checkpoint. This model is deployed as a live prototype through a documented decision of the project owner (section 10), so the complete lifecycle can be demonstrated, while the evaluation results are reported exactly as measured.

Main project objective

Predict whether an order will be Delivered or Delayed from attributes known when the order is placed.

Success is defined by more than accuracy. With 84% of orders delivered, a model that always says Delivered is already 84% accurate yet useless. We therefore require the model to rank orders better than chance, to catch a meaningful share of delayed orders, to beat a naive baseline, and to create positive value under agreed cost assumptions (gates listed in section 3).

Where to find it: artifacts/reports/business_requirements.json, artifacts/reports/data_requirements.json

2. What, where, and why

The table summarises each lifecycle stage, the code and output files that implement it, and why the stage matters.

Stage	What is done	Where	Why
Business Understanding	Define the problem, users, requirements, success gates, and continuation decision.	mlops_pipeline/business_understanding.py; artifacts/reports/business_*.json	Prevents building a technically correct solution for the wrong problem.
Data Preparation	Remove 2,433 Returned orders, validate, describe, explore, engineer 6 features, split 60/20/20, scale.	mlops_pipeline/data_stage.py; artifacts/feature_store/	Gives every model the same clean, leakage-free input.
Analysis	Train a naive baseline, Logistic Regression, Random Forest, and XGBoost; select on validation data; log to MLflow.	mlops_pipeline/model_stage.py; artifacts/mlflow/	Makes experiments fair, comparable, and reproducible.
Evaluation	Score the frozen model once on 9,514 untouched test orders; compare with the baseline, costs, and gates.	mlops_pipeline/evaluation_stage.py; artifacts/reports/evaluation_report.*	Gives an independent, honest checkpoint before deployment.
Deployment	FastAPI service, web app, Docker image, CI, tests, smoke test, live demo.	deployment/; Dockerfile; .github/workflows/	Turns the model into a controlled service people can use.
Utilization	Use through the web app and API, monitoring, feedback, support tickets, guarded retraining.	deployment/app.py; mlops_pipeline/retraining.py	Keeps the deployed system observable and maintainable.

3. Business Understanding

Delayed orders cause customer-service contacts, compensation, and lost customer trust. If operations knew at order time which orders are likely to be late, they could act early: choose a faster carrier, prioritise picking, or inform the customer. The intended users are shipping operations, customer service, and project management.

Before looking at any model score, we fixed what success means. These gates were agreed up front so that the evaluation cannot be adjusted after seeing the results:

Business success gates

Gate	Required	What it protects against
Test ROC-AUC	>= 0.60	A model that ranks orders no better than chance (0.50)
Delayed recall	>= 0.40	A model that misses most delayed orders
Weighted F1 uplift	>= 0.02 over baseline	A model that is not better than always predicting Delivered
Net benefit vs baseline	> EUR 0	A model that costs more than doing nothing
Candidate net benefit	> EUR 0	A model whose interventions cost more than they save

Continuation decision: continue to data collection. The use case is well defined and suitable for a gated prototype; real business use depends on the evidence gathered later.

Where to find it: artifacts/reports/situation_assessment.json, business_requirements.json, data_science_project_map.json, business_checkpoint.json; thresholds in mlops_pipeline/config.py

4. Data Collection, Exploration, and Preparation

The source is **50K Records.csv**: 50,000 orders with 10 columns, placed between 2022-01-01 and 2025-12-31. The pipeline fingerprints the file (SHA-256) and checks it against an input contract before processing it.

Target scope

We keep Delivered and Delayed and remove the 2,433 Returned orders, leaving 47,567 orders. Of these, 40,062 (84.2%) were delivered and 7,505 (15.8%) were delayed. This imbalance is why accuracy alone is misleading and why we track delayed recall separately.

Data-quality result

Source rows	Returned removed	Modeled rows	Blocking defects	Data sufficient
50,000	2,433	47,567	0	Yes

All ten quality checks found zero problems (missing values, duplicates, invalid dates or categories, impossible date order, delivery-day mismatches, negative costs). Shipping cost ranges from EUR 50 to EUR 399 (median EUR 121).

What the data shows

The overall delay rate is 15.8%. Below: the lowest and highest delay rate within each attribute, and a chi-square test (p above 0.05 means no evidence of a relationship with delays).

Attribute	Lowest delay rate	Highest delay rate	Chi-square p-value
Shipping mode	15.6% (Standard)	16.2% (Same Day)	0.58
Customer region	15.3% (East)	16.2% (West)	0.35
Product category	14.9% (Home)	16.3% (Electronics)	0.28
Order month	14.8% (month 12)	16.6% (month 8)	0.43
Order weekday	15.5% (Sat)	16.2% (Fri)	0.91
Shipping cost (quartiles)	15.4% (lowest 25%)	16.3% (highest 25%)	0.20

Across every order-time attribute the delay rate stays between roughly 15% and 17%: the largest gap between two groups is 1.8 percentage points, and no chi-square test finds a significant relationship (smallest p-value 0.20). This is the first sign that order-time attributes carry little information about delays, and it explains the model results in section 7.

Features and leakage controls

The model uses six features known when the order is placed: customer region, product category, shipping mode, shipping cost, order month, and order weekday. Ship date, delivery date, and delivery days are excluded because they are only known after shipping or delivery; using them would let the model see the answer. The data is split once, stratified by outcome: 28,539 training orders (4,503 delayed), 9,514 validation orders (1,501 delayed), and 9,514 test orders (1,501 delayed). Scaling is fitted on the training orders only.

Feature store

Features and targets for all three splits, the fitted scaler, and a manifest (feature order, category codes, target map, split sizes) are stored together, so the API, notebooks, and retraining use exactly the same inputs. An output contract checks column order, types, and that the target is only 0 or 1.

Where to find it: artifacts/reports/data_quality_report.json, data_exploration_report.json, data_preparation_report.json, input_data_contract.json, output_data_contract.json; artifacts/feature_store/; notebook E- commerce.ipynb

5. Analysis and Modeling

We compared four models on the same training data and judged them only on the validation data, keeping the test data untouched. The **naive baseline** always predicts the majority class (Delivered) and sets the bar every real model must clear. **Logistic Regression** is simple and easy to explain, **Random Forest** and **XGBoost** can capture non-linear patterns and interactions. Logistic Regression and Random Forest weight the classes so the rarer Delayed class is not ignored.

Validation model and business factors

Model	ROC-AUC	Accuracy	Bal. acc.	Weighted F1	Delayed recall	Net benefit	Train time
XGBoost	0.5078	0.8402	0.4994	0.7695	0.0013	EUR -104,086	9.6 s
Random Forest	0.5054	0.7748	0.5035	0.7539	0.1073	EUR -95,980	6.0 s
Logistic Regression	0.5009	0.5045	0.4985	0.5706	0.4897	EUR -70,580	0.8 s
Naive majority baseline	0.5000	0.8422	0.5000	0.7701	0.0000	EUR -104,054	0.0 s

The selection rule, fixed in advance, is the highest validation ROC-AUC among the real models. XGBoost scored 0.5078, +0.0078 above the baseline. At the default threshold of 0.50 it almost always predicts Delivered (delayed recall 0.0013), so the decision threshold is tuned next.

Choosing the decision threshold

The model outputs a probability that an order will be delivered. We tested every threshold from 0.10 to 0.90 on the validation data. The preferred rule was to pick the threshold with the highest positive net benefit while catching at least 40% of delayed orders; no threshold reached a positive net benefit, so the fallback rule picks the most balanced threshold that still catches at least 40%. That threshold is **0.85**: an order is predicted Delivered only if the model is at least 85% sure. On validation data it gives a delayed recall of 0.502 and a balanced accuracy of 0.510.

Business factors such as training time, prediction speed, interpretability, and operational complexity are recorded for every model next to its scores, because a slightly better model is not worth it if it is much harder to run or explain.

Where to find it: artifacts/reports/model_comparison_validation.csv, model_business_factors.csv, threshold_sweep_validation.csv, analytical_model_selection.json; artifacts/model_metadata_store/; MLflow; notebook modelling.ipynb/Models.ipynb

6. Cost_per_Day leakage review

An earlier notebook proposed the feature Cost_per_Day = Shipping_Cost / Delivery_Days. It looks useful, and it would probably raise the scores, but Delivery_Days is only known after the order has been delivered. At the moment we want to make the prediction, that number does not exist yet.

Using it would be like predicting the weather with tomorrow's newspaper: the score goes up, but the model could never work in practice. We therefore excluded it, together with Ship_Date, Delivery_Date, and Delivery_Days, from every split, model, and evaluation.

EXCLUDED FROM ALL MODEL SELECTION AND EVALUATION

A high feature importance or apparent score improvement would not be valid evidence; it would measure access to post-outcome information.

Where to find it: artifacts/reports/cost_per_day_leakage_review.json/.md, feature_leakage_assessment.json

7. Evaluation and business checkpoint

Final selected algorithm: XGBoost. Deployment status: deployed as a live prototype; 2 of 5 evaluation gates met.

Once the model and its threshold were frozen, we opened the test set exactly once: 9,514 orders (1,501 delayed) that played no part in training or tuning. This gives an honest estimate of how the model would behave on new orders.

Test measure	XGBoost	Naive baseline	Required	Result
ROC-AUC	0.4890	0.5000	>= 0.60	Not met
Delayed recall	0.4744	0.0000	>= 0.40	Met
Weighted F1	0.5717	0.7701	+0.02 uplift	Not met
Net benefit	EUR -72,726	EUR -104,054	> baseline	Met
Net benefit (absolute)	EUR -72,726	-	> EUR 0	Not met
Accuracy	0.5055	0.8422	-	-
Balanced accuracy	0.4928	0.5000	-	-
Delayed precision	0.1538	0.0000	-	-

How to read these results

What works. The model catches 47.4% of delayed orders, while the baseline catches none. Under the cost assumptions in section 8 this makes it EUR 31,328 cheaper than the baseline on the test set.

What does not work yet. A ROC-AUC of 0.489 means the model ranks delayed and delivered orders about as well as chance (0.50). To catch nearly half of the delayed orders it has to flag 4,628 of 9,514 orders, and only 15.4% of those flags are real delays. Accuracy and weighted F1 fall below the baseline because many delivered orders are flagged as well.

Why. Section 4 already showed that delay rates barely change across region, category, shipping mode, cost, and date. No algorithm can learn a pattern that the inputs do not contain, so the limit lies in the available data rather than in the modelling.

Checkpoint decision: RETURN_TO_BUSINESS_UNDERSTANDING. In MLOps-DSPM terms, the next iteration returns to business understanding to obtain stronger pre-shipment data (carrier, warehouse, route distance, promised delivery date, dispatch backlog) before the model is used for real decisions.

Where to find it: artifacts/reports/evaluation_report.json/.md, final_model_report.json, checkpoint_decision.json, naive_vs_best_candidate.png; notebook evalaution/stage4_evaluation.ipynb

8. Cost and benefit of correct and wrong classifications

A prediction can be right or wrong in two ways each, and each case has a different business effect. Delivered is class 1 and Delayed is class 0, so a correctly flagged delay is a true negative. The counts below are for the 9,514 test orders at threshold 0.85.

Outcome	Count	Meaning	Unit value	Total
Correct delay flag (TN)	712	Delayed order flagged; operations can intervene	+EUR 30	EUR 21,360
Correct delivered (TP)	4,097	Delivered order left alone	+EUR 2	EUR 8,194
Missed delay (FP)	789	Delayed order not flagged	-EUR 80	EUR -63,120
Unnecessary intervention (FN)	3,916	Delivered order flagged as delayed	-EUR 10	EUR -39,160
Net benefit				EUR -72,726

Of the 1,501 delayed test orders, 712 were flagged and 789 were missed. The naive baseline flags nothing, so it misses all 1,501 delays and ends at EUR -104,054. The model's EUR -72,726 is therefore EUR 31,328 better, but still negative overall, because each missed delay is assumed to cost EUR 80.

The unit values are transparent scenario assumptions set in mlops_pipeline/config.py. Finance and operations should replace them with measured customer-service, compensation, and intervention costs before any investment decision.

Business objective: pros

The pipeline is fully reproducible; the test set is isolated; the model finds 47.4% of delayed orders that would otherwise go unnoticed; it is cheaper than doing nothing under the stated assumptions; and every decision is documented and auditable.

Business objective: cons

Ranking quality is close to chance; many delivered orders are flagged, which would waste intervention effort; the absolute net benefit is negative; and the order-time inputs contain little signal about delays.

Where to find it: artifacts/reports/cost_benefit_analysis.json/.md, business_objective_assessment.json/.md

9. MLflow, metadata store, and model registry

Every model we trained is logged to MLflow with its parameters, training time, prediction speed, validation scores, estimated business value, the saved model, an input example, and its signature. The final test evaluation is logged as a separate run with all business reports attached. This means any result in this report can be traced back to the exact run that produced it.

The model registry keeps two clearly separated entries. The **candidate** is the selected model with the automatic evaluation result. The **champion** is the model that is actually allowed to serve predictions, together with who approved it, when, and why.

View MLflow

```
mlflow ui --backend-store-uri sqlite:///artifacts/mlflow/tracking.db --host 127.0.0.1 --port 5000
```

Then open http://127.0.0.1:5000 and select the experiment **ecommerce-shipping-bpmn**. The final evaluation run ID is abd77c18479045b99b8cb9b3dd73a0b2.

Current registry result

Candidate: XGBoost, automatic gates not all met (registry stage: rejected). Champion: the same evaluated XGBoost bundle, verified by its SHA-256 fingerprint and deployed as a live prototype by Vivek Kumar Singh (project owner) on 2026-10-15.

Where to find it: artifacts/mlflow/tracking.db; artifacts/model_metadata_store/*.json; artifacts/model_registry/candidate_manifest.json and champion_manifest.json

10. Deployment

Deployment means more than saving a model file. The model is packaged with its scaler, category codes, feature order, and threshold into one bundle, and served by a FastAPI service with single and batch prediction endpoints, health, model, and pipeline information, monitoring, feedback, and support. The same service hosts a web app at its root address, so users can try predictions in a browser. A Dockerfile builds a self-contained image, a GitHub Actions workflow runs the full test suite on every push, and a smoke-test script checks a running deployment end to end.

Gate-aware status

The service only serves predictions from a model that is allowed to serve: either one that passed every gate, or one with a documented deployment decision. Without either, prediction requests are refused (HTTP 503), so no unapproved model can serve unnoticed.

Deployment decision. Vivek Kumar Singh (project owner) deployed the evaluated XGBoost model as a live prototype on 2026-10-15. Reason: Course requirement: publish the evaluated model as a live prototype to demonstrate the complete MLOps pipeline end to end. Gates not yet met: net benefit > EUR 0, test ROC-AUC >= 0.60, weighted F1 uplift >= 0.02. Each API response states its deployment basis and notes that the model is a prototype.

Live deployment smoke test: PASSED on 2026-10-15. A real prediction was served (HTTP 200), and health, pipeline status, monitoring, and metrics were all available.

Live demo

https://huggingface.co/spaces/VivekSinghK14/shipping-delay-predictor - a free Hugging Face Space. The trained XGBoost model is exported to JSON (scripts/export_browser_model.py) and runs in the browser. An automated test confirms it gives the same predictions as the Python model on all 47,567 orders.

Prototype commands

```
uvicorn deployment.app:app --host 127.0.0.1 --port 8000
python scripts/smoke_test_deployment.py --base-url http://127.0.0.1:8000 --require-approved
docker build -t ecommerce-shipping-prediction .
docker run --rm -p 8000:8000 ecommerce-shipping-prediction
```

Where to find it: deployment/app.py, deployment/static/, deployment/static_space/, Dockerfile, .github/workflows/deployment-ci.yml, artifacts/reports/deployment_smoke_test.json, deployment_override.json

11. Utilization, monitoring, and retraining

Users enter an order (region, category, shipping mode, cost, and date) in the web app and immediately see Delivered or Delayed with the probability. Technical users can call the same API through the Swagger page at /docs. Every prediction gets an ID, so the real outcome can be reported later through /feedback.

The service records requests, predictions, errors, the share of each predicted class, the average predicted probability, and recent events. Once real outcomes arrive, it computes observed accuracy, ROC-AUC, and delayed recall, and recommends retraining when at least 30 outcomes show ROC-AUC below 0.60 or delayed recall below 0.40. Metrics are also exposed in Prometheus format for dashboards.

Retraining workflow

Retraining starts when the source data changes (detected by its fingerprint), when the model is older than 30 days, or when the observed ROC-AUC reported to the retraining script falls below 0.60, and it can be forced manually. Each attempt runs the full pipeline in its own timestamped folder, and the current champion is only replaced if the new model passes every gate.

```
python scripts/retrain_model.py --force
```

Retraining run 20261015T113019Z (triggers: no_previous_retraining_state, manual_force) completed with decision RETURN_TO_BUSINESS_UNDERSTANDING and was not promoted, so the running champion stayed unchanged, exactly as designed.

Utilization status: active as a live prototype (web app, API, and monitoring).

Where to find it: /monitoring, /metrics, /utilization, /feedback in deployment/app.py; runtime/prediction_log.jsonl; artifacts/retraining_report.json; artifacts/retraining_runs/

12. Testing, CI, and reproducibility

Automated tests protect every stage of the pipeline and the deployed service, so a change that breaks something is caught before it reaches users.

Test file	Tests	What it checks
test_bpmn_pipeline.py	11	Data stage outputs and split sizes, Returned removal, leakage exclusions, cost model, checkpoint report, retraining trigger, business artifacts, lifecycle status, deployment decision record
test_browser_model.py	1	Live web app predictions match the Python model on all 47,567 orders
test_deployment_api.py	12	Health, single/batch/processed predictions, blocking without approval, invalid input, monitoring, metrics, model info, feedback, support tickets, web app, deployment decision

Current result: all 24 tests pass.

The GitHub Actions workflow installs the runtime and development dependencies and runs the complete suite on every push and pull request.

Reproduce the project

```
pip install -r requirements.txt -r requirements-dev.txt
python scripts/run_end_to_end_pipeline.py
pytest -q
python scripts/generate_project_report.py
```

The pipeline uses a fixed random seed (42), so rerunning it reproduces the same splits, models, and numbers. Add --quick for a faster structural run with smaller models.

Where to find it: tests/; .github/workflows/deployment-ci.yml; README.md

13. Requirement traceability

Each requirement from the project proposal and the MLOps-DSPM material, with the evidence that shows it is done.

Requirement	Implementation evidence	Status
Business Understanding	Situation assessment, requirements, project map, success gates (section 3)	Complete
Data exploration and preparation	Quality checks, contracts, 2,433 Returned removed, feature store (section 4)	Complete
Model selection and training	Four models, validation-only selection, MLflow tracking (section 5)	Complete
Leakage review	Cost_per_Day and post-outcome fields excluded (section 6)	Complete
Business-goal evaluation	Untouched test set, costs, 2 of 5 gates met (sections 7-8)	Complete
Deployment and testing	FastAPI, web app, Docker, CI, 24 tests, smoke test (sections 10, 12)	Live prototype
Use, monitoring, maintenance	Monitoring, feedback, support, guarded retraining (section 11)	Active (prototype)
Architecture documentation	BPMN-aligned process, UML component and sequence diagrams (docs/architecture.md)	Complete
Critical reflection	Data, modelling, technology, and process review (section 14)	Complete

14. Challenges and critical reflection

Data challenge

The dataset is clean, but clean data is not the same as predictive data. Delay rates are almost identical across every order-time attribute, so the model has very little to learn from. The most valuable next step is to collect information that is known before shipping and plausibly drives delays: carrier, warehouse, route distance, promised delivery date, stock availability, dispatch backlog, and weather.

Modeling challenge

The earlier notebooks mixed in Returned orders as a third class and tuned the decision threshold on the test set, which makes results look better than they are. The corrected design removes Returned completely, models Delivered vs Delayed only, keeps training, validation, and test strictly separate, and compares all models on the same terms.

Technology reflection

CSV and Joblib files for the feature store and a local SQLite MLflow backend fit an academic prototype well: they are simple, transparent, and free. A production system would use a managed feature store, a shared MLflow server, central artifact storage, orchestration, authentication, and alerting. For the public demo, a free static Hugging Face Space runs the exported model in the browser; a hosted server would add shared monitoring.

Process-model reflection

The MLOps-DSPM is valuable because it makes evidence-based decisions possible, including the decision to go back a stage. Here the pipeline, deployment, monitoring, and retraining are complete and running, and the evaluation clearly shows what is needed next: better input data. The model is deployed as a documented live prototype, so the complete lifecycle can be demonstrated while the evidence and the decision stay transparent.

15. Final conclusion

The project implements the complete MLOps-DSPM lifecycle required by the proposal: management artifacts, a reproducible data pipeline with a feature store, tracked experiments in MLflow, a model registry, a business-focused evaluation, a deployed API and web app, monitoring, 24 automated tests with CI, and guarded retraining.

The final selected algorithm is XGBoost. On 9,514 unseen test orders it catches 47.4% of delayed orders and is EUR 31,328 cheaper than doing nothing, meeting 2 of 5 gates. Its ROC-AUC of 0.489 shows that order-time attributes alone cannot yet predict delays reliably. The model is live as a prototype for demonstration. The next iteration should confirm the intervention process and true costs with the business and collect stronger pre-shipment operational data.

Final status

LIVE PROTOTYPE DEPLOYED - 2 OF 5 EVALUATION GATES MET

References

1. Machine Learning-Operations-based Data Science Process Model - Material (DSPM_material_STP.pdf).
2. Project Proposal: Development and Operationalization of Data Science Solutions - E-Commerce Shipping Prediction (STP_ecommerce.pdf).
3. Kreuzberger, D., Kuhl, N., and Hirschl, S. (2023). Machine Learning Operations (MLOps): Overview, Definition, and Architecture. IEEE Access, 11, 31866-31879.
4. Haertel, C., Pohl, M., Nahhas, A., Staegemann, D., and Turowski, K. (2022). Toward a Lifecycle for Data Science. PACIS 2022 Proceedings.